PostgreSQL Migration

ABSTRACT
This course deals with data migration from other RDBMS to PostgreSQL. PostgreSQL to PostgreSQL and Cloud Migration

[Author name]
Mohammed Abdul Sami

Table of content

Manual Migration method	3
PostgreSQL to PostgreSQL Migration	5
Tools Based Migration	6
Oracle to PostgreSQL Migration Challenges	7
Migration to PostgreSQL RDS	9
T-SQL and PL/pgSQL Language Differences	12
SQLine tool	14

Database Migrations

Database migrations means moving data from database platform to another for various reasons like leveraging could platforms or migrating to open source platforms like postgresql or mysql or traditionally when the hardware is upgraded to newer hardware.

Migration Scenarios

1. Other databases to PostgreSQL
2. PostgreSQL to PostgreSQL
3. Migration to Cloud based PostgreSQL

The above mentioned are scenarios where data is migrated to PostgreSQL. Usually, Migration is a complex process involves not just data migration but also includes migrating schema, user and grants data etc.

Migration Methods

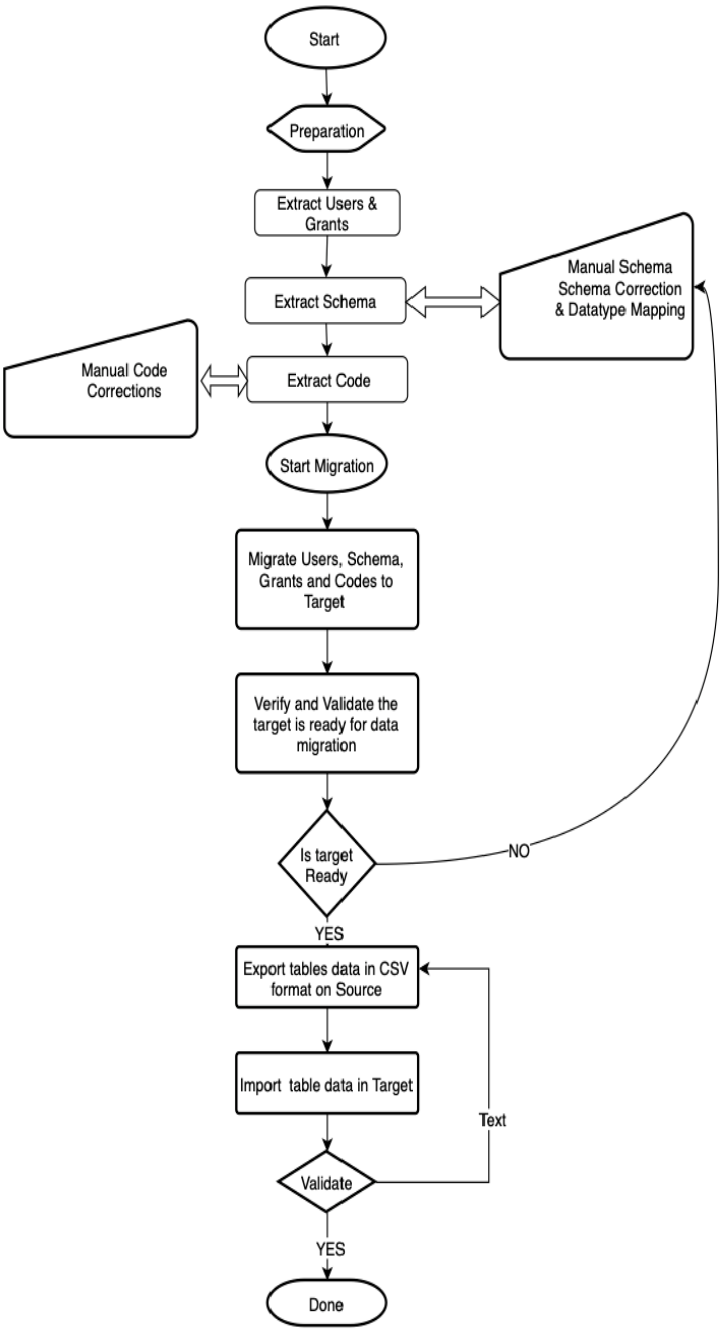
1. Manual Migration method
2. Tools based migration
3. Using cloud based Migration tools called DMS

For internal use only not for redistribution

Manual Migration method

Manual Migration method involves various activities performed by DBA's and Developers to migrate the Schema, Roles & Grants, Backend Code and Data

Typically following flowchart illustrates the flow of migration



For internal use only not for redistribution

Steps Involves:

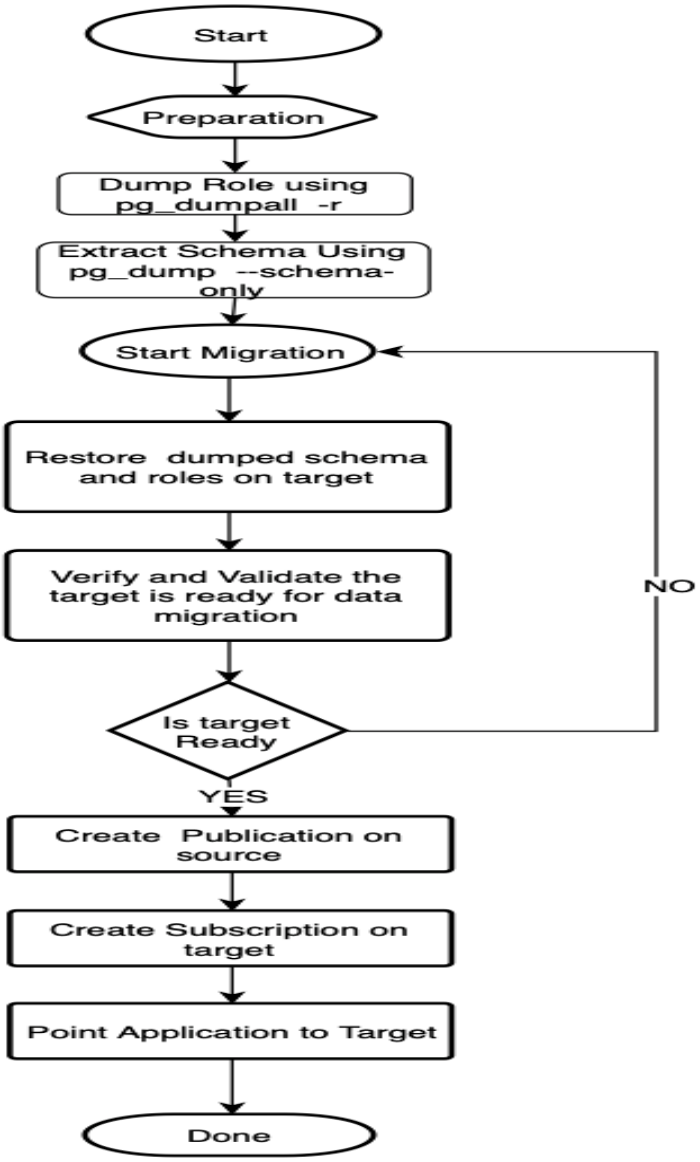
1. Extract User & Grant info from Source
2. Extract Schema
3. Make correction to schema as needed by PostgreSQL
4. Extract Code
5. Make Correction to code acceptable to PostgreSQL
6. Create the extracted objects on target PostgreSQL and validate
7. Prepare for the cutoff
8. During Cutoff export data from source server in CSV format
9. Import Data on target PostgreSQL
10. validate if everything is OK and point the application to PostgreSQL

For internal use only not for redistribution

PostgreSQL to PostgreSQL Migration

Sometime there is a need to migrate data from Postgres to Postgres due the fact that database is moved to new hardware, Migration to Cloud based Postgres or version upgrade done on a new server.

Following flow chart illustrate steps needed



For internal use only not for redistribution

PostgreSQL to PostgreSQL migration can be done easily using Logical Replication of PostgreSQL as logical replication can be setup between different versions of PostgreSQL.

In first step migrate all ROLES, SCHEMA and CODE using pg_dump & pg_dumpall tools

and then create a publication on source for TABLES ALL and corresponding subscription on target server.

Once the tables are populated cutoff the application to target.

Tools Based Migration

ora2pg tool

A free Perl based tool to migrate Oracle database to PostgreSQL. It can perform the following things. It generates necessary sql code to be executed in PostgreSQL to import.

Type of objects can be exported (One at a time):

- TABLE: Extract all tables with indexes, primary keys, unique keys, foreign keys and check constraints.
- VIEW: Extract only views.
- GRANT: Extract roles converted to Pg groups, users and grants on all objects.
- SEQUENCE: Extract all sequence and their last position.
- TABLESPACE: Extract storage spaces for tables and indexes (Pg >= v8).
- TRIGGER: Extract triggers defined following actions.
- FUNCTION: Extract functions.
- PROCEDURE: Extract procedures.
- PACKAGE: Extract packages and package bodies.
- INSERT: Extract data as INSERT statement.
- COPY: Extract data as COPY statement.
- PARTITION: Extract range and list Oracle partitions with subpartitions.
- TYPE: Extract user defined Oracle type.
- FDW: Export Oracle tables as foreign table for oracle_fdw.
- MVIEW: Export materialized view.
- QUERY: Try to automatically convert Oracle SQL queries.
- SYNONYM: Export Oracle's synonyms as views on other schema's objects.
- DIRECTORY: Export Oracle's directories as external_file extension objects.

All the above objects can be exported from Oracle database one at a time or all once however following object needs manual verification before being imported into PostgreSQL

- GRANT
- TABLESPACE

- TRIGGER
- FUNCTION
- PROCEDURE
- TYPE
- QUERY
- PACKAGE

Commercial Tools

Following are the Commercial Migration tools available for Oracle to PostreSQL Migration

- EDB Migration Tool
- 2nd Quadrant Migration Services

Oracle to PostgreSQL Migration Challenges

it's important to understand that no tool that can automate the process completely. Migration requires a hands-on approach.

Following pointers are handy while performing migration

1 Preliminary Checking

Make sure you understand what needs to be migrated. This requires systematic mapping and checking. Also, look for any historical data that you may need. Don't waste time replicating unnecessary data. For example, this can be backup data and temporary tables from past maintenance procedures.

2 Migration Assessment

Once you have completed your preliminary checking, you need to analyze app and database objects to locate incompatibilities between the databases.

Tools such as Ora2pg are very helpful with migration assessment. They hook up seamlessly to the Oracle database, scan it automatically and extract the data to generate a comprehensive database migration report. To sum it up, this step will help you estimate the time and cost required for the migration process.

3 Data Type Mapping

Oracle and PostgreSQL have quite a lot of differences when it comes to data types. You need to do some mapping and make sure things are being done correctly.

Oracle	PostgreSQL
VARCHAR2(n)	VARCHAR(n)
CHAR(n)	CHAR(n)
NUMBER(n,m)	NUMERIC(n,m)
NUMBER(4)	SMALLINT

For internal use only not for redistribution

NUMBER(9)	INT
NUMBER(18)	BIGINT
NUMBER(n)	NUMERIC(n)
DATE	TIMESTAMP(0)
TIMESTAMP WITH LOCAL TIME ZONE	TIMESTAMPTZ
CLOB	TEXT
BLOB RAW(n)	BYTEA(1 GB limit) Large object

4 Watch the Transactions

Oracle has built-in transaction functionality, which has to be activated manually in PostgreSQL. In Oracle, the transaction starts when executing any statement and ends with the COMMIT statement. In PostgreSQL, the transaction commences with BEGIN and ends with the COMMIT statement.

PostgreSQL possesses all the isolation levels just like Oracle. However, keep in mind that the default isolation level of PostgreSQL is Read Committed.

5 Don't Ignore the Differences

There are quite a few technical issues when it comes to migrating Oracle to PostgreSQL. You need to make sure your team is aware of the following:

Dual Table - In Oracle, the FROM clause is mandatory for every SELECT statement. Hence, Oracle database uses DUAL table for SELECT statements where the table name is not required. In PostgreSQL, the FROM clause is not mandatory. Hence, DUAL table is not required.

Empty String and NULL - There is a clear difference in behavior when it comes to these two important parameters. In Oracle, the IS NULL operator is used to check whether a string is empty or not. However, in PostgreSQL, the result is FALSE for empty string and TRUE for NULL.

SUBSTR - SUBSTR behaves differently in Oracle and PostgreSQL. It works smoothly in PostgreSQL but returns a different result in Oracle. This difference can lead to potential issues. The solution is to use the Oracle SUBSTR function which returns the same result as Oracle.

The DELETE Statement - In Oracle, the DELETE statement can work without the FROM clause, which is not the case in PostgreSQL. You will need to add the FROM clause in PostgreSQL DELETE statement manually. Ignoring this step can cause a plethora of issues going ahead.

6 PLSQL to PLPGSQL Conversion

- PostgreSQL's PL/pgSQL language is similar to Oracle's PL/SQL language in many ways. It is a block-structured, imperative language, and all variables have to be declared. Assignments, loops, conditionals are similar in both databases. However, the main differences you should keep in mind while porting are:

For internal use only not for redistribution

- In PostgreSQL, the function body must be written as a string literal. You need to use a dollar quoting or escape single quotes within the function body.
- There are no packages in PostgreSQL. Instead, use schemas to organize your functions into the required/ desired groups.
- Data type names often need translation. For example, in Oracle string values are commonly declared as “varchar2”, a non-SQL-standard type.
- There are various notational differences for the use of cursor variables.
- Since there are no packages, there are no package-level variables either. You can keep per-session state in temporary tables instead.

7 Testing

Testing the newly the migrated database is crucial because despite many functions being the same in both databases, the outcome can turn out to be extremely different in PostgreSQL. You probably don’t have unlimited time for this, but the following scenarios must be tested to ensure smooth performance.

- Make sure all objects are correctly converted.
- Check whether all the DMLS’s are working properly.
- Load some sample data in both databases to check the results. The SQL results from both databases should be identical.
- Assess the DML performance and work on it if necessary.

Migration to PostgreSQL RDS

Migration to aws rds instances can be done using amazon provided DMS (data migration services) which gives flexibility of migrating any of the supported databases to PostgreSQL RDS.

How it works?

Source data must be accessible from amazon cloud.

Components

For internal use only not for redistribution

1) End Points
2) AWS Replication Instant
3) Migration Task

End Points:

End Points determine the connections to source and target servers.

AWS Replication Instant:

Replication Instant facilitates data transfer from source to target and also it performs any changes needed to data.

Migration Task:

Migrations task allows control when to start or stop migration. Migration task can be created in a such way that it start immediately after creating it or it can be started any time afterward.

Procedure

1. Create End Points
2. Create Replication Instant
3. Create Migration Task
4. Run Migration task

DMS allows us to transfer the schemas and data one time or it also allows us to continuously replicate while the production server runs until we are ready for cutoff. Continuous replication minimizes downtime.

Magically DMS allows to replicate data from any source RDBMS to PostgreSQL.

SQL Server to PostgreSQL Migration

differences between SQL Server and PostgreSQL datatypes:

Numeric Datatypes

by: Mohammed Abdul Sami
For internal use only not for redistribution

Datatype	MS SQL Server	PostgreSQL
Integer datatype	INT	INTEGER
Big Integer datatype	BIGINT	BIGINT
Small Integer	SMALLINT	SMALLINT
Decimal	DECIMAL(p, s)	DECIMAL(p, s)
Floating Point	FLOAT(n)	FLOAT(n)
Real Numbers	REAL	REAL

String and Text Datatypes

Datatype	MS SQL Server	PostgreSQL
Fixed-length string	CHAR(n)	CHAR(n)
Variable-length string	VARCHAR(n)	VARCHAR(n)
Text	TEXT	TEXT
Unicode	NVARCHAR(n)	VARCHAR(n)

Date and Time Datatypes

Datatype	SQL Server	PostgreSQL
Date	DATE	DATE
Time	TIME	TIME
Timestamp	DATETIME, DATETIME2	TIMESTAMP
Time with time zone	DATETIMEOFFSET	TIMESTAMPTZ

Boolean and Binary Datatypes

Datatype	MS SQL Server	PostgreSQL
Boolean	BIT	BOOLEAN
Binary Data	VARBINARY(n)	BYTEA

For internal use only not for redistribution

Other Special Datatypes

Datatype	MS SQL Server	PostgreSQL
XML	XML	XML
JSON	NVARCHAR, JSON	JSON, JSONB
UUID	UNIQUEIDENTIFIER	UUID
Array	Not Supported	ARRAY

T-SQL and PL/pgSQL Language Differences

Block Structure

T-SQL

```
CREATE FUNCTION function_name (@parameter datatype)
RETURNS datatype
AS
BEGIN
 -- Statements
 RETURN value;
END
```

PL/pgSQL

```
CREATE FUNCTION function_name (parameter datatype)
RETURNS datatype AS $$
DECLARE
  -- Variable declarations
BEGIN
  -- Statements
  RETURN value;
END;
$$ LANGUAGE plpgsql;
```

Variables

T-SQL requires variables to be prefixed with @ and PL/pgSQL don't need it.

by: Mohammed Abdul Sami
For internal use only not for redistribution

T-SQL

```
DECLARE @variable_name datatype;
```

```
PL/pgSQL
DECLARE
  variable_name datatype;
```

```
IF Condition Syntax:
T-SQL
IF condition
BEGIN
  -- Statements
END
ELSE
BEGIN
  -- Statements
END
```

PL/pgSQL

```
IF condition THEN
  -- Statements
ELSE
  -- Statements
END IF;
```

Looping

T-SQL

```
WHILE condition
BEGIN
  -- Statements
END
```

PL/pgSQL

```
WHILE condition LOOP
  -- Statements
END LOOP;
```


by: Mohammed Abdul Sami
For internal use only not for redistribution

Error (Exception) Handling

T-SQL

```
BEGIN TRY
  -- Statements
END TRY
BEGIN CATCH
  -- Error handling
END CATCH
```

PL/pgSQL

```
BEGIN
  -- Statements
EXCEPTION
  WHEN others THEN
    -- Error handling
END;
```

SQLine tool

SQLines is a tool designed to facilitate the migration of databases from one platform to another, such as from SQL Server to PostgreSQL.

Schema Conversion: SQLines converts the database schema (DDL) from the source database (e.g., SQL Server) to the target database (e.g., PostgreSQL). This includes tables, indexes, constraints, and data types.

SQL Code Conversion: The tool translates SQL code, including queries, views, and DML statements, from the source database to the target database. It handles differences in SQL syntax and functions between the two platforms.

Stored Procedures and Functions: SQLines converts stored procedures, functions, and triggers from the source database to the target database. This may involve rewriting some parts of the code to accommodate the target database's syntax and features.

Data Migration: SQLines Data, a component of SQLines, helps transfer data from the source database to the target database. It ensures data integrity and consistency during the migration process.

Command Line and API: SQLines provides command-line interfaces and APIs for automation and integration into migration workflows. This allows for batch processing and customization of the migration process.

For internal use only not for redistribution